

End-to-End Test Guide

Operations & Finance Management System — manual UAT, role by role

Purpose: exercise every major feature, by role, and confirm it works.

Audience: a manual tester with no prior knowledge of the system.

Surfaces: API (:3000) · Admin console · Web app · Mobile (Flutter).

Legend: **EXPECT** ✓ should succeed/appear · **MUST BLOCK** ✗ negative test (blocking = pass) · ☐ tick per case.

Prepared: 28 July 2026

AES Operations & Finance — End-to-End Manual Test Plan

Audience: a manual tester (QA) with no prior knowledge of the system. **Goal:** exercise every major feature, by role, and confirm it works — while building a mental model of the platform. **How to use:** work top-to-bottom. Each test case has a **Role**, **Preconditions**, numbered **Steps**, an **Expected result**, and a **Pass/Fail** box. Record actual behaviour + a screenshot for anything that fails.

The platform has three web/desktop surfaces + one mobile app, all talking to one API: - **API** (backend) — REST at <http://localhost:3000>, interactive docs at <http://localhost:3000/docs>. - **Admin console** — configuration + back-office (React). All admin/config lives here. - **Web app** — the everyday end-user/field + finance workstation (React). - **Mobile app** — Flutter field app (Android), offline-tolerant.

1. Environment setup

Start these in order (see [README.md](#) / the local-startup notes for detail):

#	Component	Command (from repo root)	URL
1	Infra (Postgres/Redis/MinIO)	<code>docker compose -f docker-compose.dev.yml up -d</code>	Postgres :5433
2	API	<code>npm run api</code>	http://localhost:3000 (docs at /docs , health at /health)
3	Admin console	<code>npm --prefix apps/admin run dev</code>	http://localhost:5174 (or :5175 if 5174 is taken — watch the terminal output)
4	Web app	<code>npm run web</code>	http://localhost:5173
5	Mobile (optional)	Android emulator → <code>flutter run --flavor dev -t lib/main_dev.dart --dart-define=API_BASE_URL=http://10.0.2.2:3000</code> (from <code>apps/mobile</code>)	on the emulator

Confirm ready: GET <http://localhost:3000/health> returns `{"status":"ok", ..., "database": {"status":"up"}}`.

Optional — load the demo dataset (recommended before financial tests): sign in to the admin console as **admin**, open **Configuration** → **Data Import**, choose **All workbooks**, click **Import**, then click **Re-run** on *Migration parity*. This loads 10 orders, contracts, expenses, loans, payroll and lights up every dashboard (see TC-4.x).

2. Test accounts

All accounts are pre-created. Passwords below. (@[aes.local](#) domain.)

Role	Email	Password	Scope	What they represent
System Administrator	admin@aes.local	ChangeMe!123	Global	Configures everything; no approval/finance authority over own requests
Finance Director (FD)	fd@aes.local	FdPassword!123	Global	Approves finance items, payroll, petty cash > threshold; tax oversight
Finance Officer	fo@aes.local	AesTest!234	Global	Transaction entry, receipts/payments, petty cash floats, payroll drafts, reports
	od@aes.local	AesTest!234	Global	

Role	Email	Password	Scope	What they represent
Operations Director (OD)				Co-authorises budgets, approves operational requisitions
Director	director@aes.local	AesTest!234	Global	Director withdrawals; sees director-confidential boards; danger alerts
Site Manager	sitemgr@aes.local	AesTest!234	Mimosa	First-line approver of site timesheets/requests; site dashboards
Site Clerk / Timekeeper	clerk@aes.local	AesTest!234	Mimosa	Captures timesheets + petty-cash vouchers; raises requisitions
Operations Staff	opsstaff@aes.local	AesTest!234	Global	Creates orders/expenses; raises requisitions/travel
Auditor	auditor@aes.local	AesTest!234	Global	Read-everything, write-nothing (external accountant)

Site scoping: `sitemgr` and `clerk` are scoped to **Mimosa** — they should only see Mimosa data, never Unki/Zimplats. That scoping is itself a test (TC-6.4).

3. How to record results

For each test case tick **PASS** or **FAIL**. On **FAIL**, note: what you did, what you expected, what actually happened, and attach a screenshot. "N/A" if a precondition couldn't be met.

Legend for expected outcomes: **EXPECT** ✓ = should succeed/appear · **MUST BLOCK** ✗ = should be blocked/hidden (a *negative* test — blocking is the pass).

4. Quick role × area access matrix (orientation)

Area	SysAdmin	FD	FinOfficer	OD	Director	SiteMgr	Clerk	Op
Admin config (users/rates/matrix)	EXPECT ✓	partial	–	–	–	–	–	
Financial data entry (orders/expenses)	EXPECT ✓	EXPECT ✓	EXPECT ✓	view	view	–	–	EXP
Approvals inbox	EXPECT ✓	EXPECT ✓	–	EXPECT ✓	EXPECT ✓	EXPECT ✓	–	
Requisitions/Travel (raise)	EXPECT ✓	EXPECT ✓	EXPECT ✓	EXPECT ✓	EXPECT ✓	EXPECT ✓	EXPECT ✓	EXP
Petty cash (transact)	–	approve	EXPECT ✓	–	–	confirm	EXPECT ✓	
Payroll	EXPECT ✓	approve	draft	–	–	–	–	
Command Centre	EXPECT ✓	EXPECT ✓	–	EXPECT ✓	EXPECT ✓	OK	–	
Director-confidential boards	MUST BLOCK ✗ content	EXPECT ✓	MUST BLOCK ✗	EXPECT ✓	EXPECT ✓	MUST BLOCK ✗	MUST BLOCK ✗	MUST

Area	SysAdmin	FD	FinOfficer	OD	Director	SiteMgr	Clerk	Op
SHE / Site Reports	EXPECT ✓	–	–	EXPECT ✓	–	EXPECT ✓	capture	

(This is a guide; the individual test cases below are authoritative.)

5. Workflows & responsibilities — who carries out each activity

The universal shape. Every money / approval workflow is: a **raiser** creates and **submits** → one or more **approvers** decide → a **finance** actor **settles** (disburses / posts). Two rules hold everywhere: - **Approvals happen in the My Approvals inbox — not on the module's own page.** A module page (e.g. *Workflows* → *Requisitions*) only offers **raise** and, after approval, **settle** (Disburse) — it never shows an Approve button. Every module's approvals (requisitions, travel, budgets, petty cash, director withdrawals, payroll, timesheets) land in the one **My Approvals** queue. - **No self-approval (segregation of duties).** Whoever raises an item cannot approve or settle it.

Which app each role works in

Role	Primary app	Also uses
SysAdmin, Finance Director, Finance Officer, Ops Director	Admin console (:5175)	—
Director	Admin + Web	Mobile
Site Manager, Site Clerk, Ops Staff	Web (:5173)	Mobile (field capture)
Auditor	Admin / Web (read-only)	—

(*My Approvals* exists in every app a role can log into. The cash-position + danger-alert panels appear on the admin Dashboard and in the web/mobile Command Centre.)

5.0 At-a-glance — who raises, who approves, who settles

Workflow	Raises	Approves (in My Approvals)	Settles / executes
Cash requisition	Ops Staff / Clerk / Site Mgr	Site Manager → Finance Director (<i>only if</i> ≥ \$5,000)	Finance Officer — Disburse
Travel & allowances	Ops Staff / Site Mgr	Site Manager → Finance Director (<i>if over band</i>)	Finance Officer — Disburse then Retire
Petty-cash withdrawal	Site Clerk	Site Manager (<i>below threshold</i>) or Finance Director (<i>≥ threshold</i>)	posts on approval
Petty-cash float / conversion / count	Finance Officer	— (<i>FD unlocks a locked float</i>)	Finance Officer
Budget	Finance Officer	Ops Director and Finance Director (<i>parallel — both</i>)	Active once both approve
Director withdrawal	Director	Finance Director / another Director	Finance Officer — Complete transfer
Timesheet	Site Clerk / Timekeeper	Site Manager (<i>approve → lock</i>)	—
Payroll → returns	Finance Officer (<i>draft / run</i>)	Finance Director (<i>approve</i>)	Finance Officer (<i>file returns</i>)
Back-pay / Acting	Finance Officer	— (<i>feeds next payroll</i>)	next payroll run
Order → revenue	Ops Staff / Finance	— (<i>no approval</i>)	revenue posts when order served

5.1 Cash requisition — *raise* → *approve* → *disburse*

#	Who	Where	Activity
1	Requester (Ops Staff / Clerk / Site Mgr)	Web or Mobile → Requests → Requisition	Create draft (purpose, amount, currency, required-by, optional receipt) → Submit for approval
2	Site Manager	My Approvals	Approve — step 1 (any amount)
3	Finance Director	Admin → My Approvals	Approve — step 2, only if amount ≥ \$5,000
4	<i>system</i>	—	Funds check → Approved-Ready-to-Pay (funded) or Approved-Pending-Funds (short)
5	Finance Officer (or FD)	Admin → Workflows → Requisitions	Disburse : choose source account + reference → posts a balanced ledger journal → Closed

Lifecycle: Draft → Submitted → Approved (Ready-to-Pay / Pending-Funds) → Disbursed → Closed. **SoD:** the requester cannot disburse their own requisition; the amount decides whether the Finance Director is involved (< \$5,000 = site-manager approval only).

5.2 Travel & allowances — *raise* → *approve* → *disburse* → *retire*

#	Who	Where	Activity
1	Requester (Ops Staff / Site Mgr)	Web/Mobile → Requests → Travel	Destination + dates → per-diem auto-computed → Submit
2	Site Manager → Finance Director (if over band)	My Approvals	Approve
3	Finance Officer	Admin → Workflows → Travel	Disburse the advance
4	Finance Officer	Admin → Workflows → Travel	Retire : enter actual spend → reconciles refund-due (underspend) or overspend owed to the traveller

5.3 Petty cash

#	Who	Where	Activity
1	Finance Officer	Admin → Workflows → Petty Cash	Create a float (one per site + currency)
2	Site Clerk	Web/Mobile → Petty Cash	Record a withdrawal (+ receipt)
3a	Site Manager	My Approvals	Confirm if below the FD threshold → posts immediately
3b	Finance Director	My Approvals	Approve if ≥ threshold → then posts
4	Finance Officer	Admin → Workflows → Petty Cash	Record USD↔ZWG conversions and cash counts ; an out-of-tolerance count locks the float
5	Finance Director	Admin → Workflows → Petty Cash	Unlock a locked float

5.4 Budget — *parallel dual authorisation*

#	Who	Where	Activity
1	Finance Officer	Admin → Workflows → Budgets	Create budget + line items → Submit
2	Ops Director and Finance Director	My Approvals	Approve (parallel — both required)
3	<i>system</i>	—	

#	Who	Where	Activity
			Active only after both approve; a Return by either restarts the cycle. Actual spend tracks % consumed per line

5.5 Director withdrawal

#	Who	Where	Activity
1	Director	Admin/Web → Workflows → Director Withdrawals	Raise (amount, destination, reason) → Submit
2	Finance Director or another Director	My Approvals	Co-approve
3	system	—	Posts to ledger as " Posted — Awaiting Transfer " (shows in cash position immediately)
4	Finance Officer	Admin → Workflows → Director Withdrawals	Select transfer method (EFT/RTGS) + reference → Complete

5.6 Timesheets

#	Who	Where	Activity
1	Site Clerk / Timekeeper	Web / Mobile (offline-capable) → Timesheets	Capture the daily grid → Submit
2	Site Manager	My Approvals / Timesheets	Approve → the period Locks
3	Site Manager / Finance	Admin → Timesheets	Reopen requires an explicit unlock before any edit

5.7 Payroll → statutory returns

#	Who	Where	Activity
1	Finance Officer	Admin → People & Payroll → Payroll	Draft / Run payroll
2	Finance Director	Admin → My Approvals / Payroll	Approve the run
3	system	—	Payslips + bank schedule produced; statutory obligations auto-created
4	Finance Officer / FD	Admin → Finance → Returns Hub	Track & mark statutory returns filed

5.8 Back-pay & Acting allowances

#	Who	Where	Activity
1	Finance Officer	Admin → Finance → Payroll Adjustments	Raise a back-pay batch or acting allowance → Submit
2	system	—	Feeds the next payroll run (recomputes affected earnings)

5.9 Order → revenue (no approval chain)

#	Who	Where	Activity
1	Ops Staff / Finance	Admin → Sales & CRM → Orders	Create orders, expenses, contract claims
2	Ops / Finance	Orders	Mark an order served → revenue posts to the ledger; contract claims post receivables

5.10 Collaboration flows (*no approval chain*)

Workflow	Who	Where	Notes
Boards (Kanban)	anyone	Web/Mobile → Boards	Director-confidential boards are hidden from non-directors <i>server-side</i>
Projects / WBS	Site Manager / Ops Staff	Web + Mobile-light	Build the tree, update % progress → rolls up
Site reports / SHE	Site Manager / Clerk	Web / Admin → Site Reports, SHE	Monthly capture; SHE is structured (incidents, inspections)
CRM / Marketing	Ops Staff / Finance	Web / Admin → Business Development, Marketing	Pipeline → convert to client; campaign funnel

Tip for testers: the step-by-step test cases for these flows are in **Part E** (cash workflows), **Part H** (timesheets), **Part I** (payroll/returns), **Part J** (back-pay/acting) and **Parts K–N** (collaboration). This section is the map; those are the drills.

PART A — Authentication & RBAC

TC-A.1 — Login (happy path)

Role: any · **Where:** Web app (:5173) 1. Open the web app. 2. Enter `fd@aes.local` / `FdPassword!123`. 3. Click **Sign in**. **Expected:** **EXPECT** ✓ Redirected to Home; the user's email + role chip is shown. **API:** `reachable`. ☐ PASS ☐ FAIL — Notes:

TC-A.2 — Login (wrong password)

Role: any 1. Log in with `fd@aes.local` / `wrongpass`. **Expected:** **MUST BLOCK** ✗ Rejected with an error; not signed in. ☐ PASS ☐ FAIL

TC-A.3 — Session persistence / refresh

1. Log in. 2. Refresh the browser (F5). 3. Wait > 15 min and perform an action. **Expected:** **EXPECT** ✓ Still signed in after refresh; the access token silently refreshes (no forced re-login mid-session). ☐ PASS ☐ FAIL

TC-A.4 — Logout

1. Use the top-right menu → **Sign out**. **Expected:** **EXPECT** ✓ Returned to login; protected pages redirect to login. ☐ PASS ☐ FAIL

TC-A.5 — Navigation is role-gated (do this for EACH role)

For each account, log into **both** the admin console and the web app and note the visible navigation. **Expected:** **EXPECT** ✓ Menus differ by role. e.g. `clerk` sees Timesheets/Requests but **not** Payroll/Danger Rules; `auditor` sees read-only views; `admin` sees the full Configuration group; `opsstaff` sees Orders/Requests but not admin config. ☐ PASS ☐ FAIL — record what each role sees:

PART B — System Administrator configuration (`admin@aes.local`)

All in the Admin console.

TC-B.1 — Users & Roles

1. **Configuration** → **Users & Roles**. 2. Create a user `temp1@aes.local` with a role (e.g. OPS_STAFF). 3. Assign a second role scoped to a site. 4. Verify it appears in the list. **Expected:** **EXPECT** ✓ User created; roles (incl. site scope) shown. Log out and confirm `temp1` can log in. ☐ PASS ☐ FAIL

TC-B.2 — Sites

1. **Configuration** → **Sites**. 2. Add a site (e.g. "Test Mine", type MINE_SITE). 3. Toggle active. **Expected:** **EXPECT** ✓ Site created and editable. ☐ PASS ☐ FAIL

TC-B.3 — Exchange rates (effective-dated, append-only)

1. **Configuration** → **Exchange Rates**. 2. Add a USD/ZWG rate for today (official + parallel). 3. Add a second one tomorrow. **Expected:** **EXPECT** ✓ Both rows kept (history preserved; old rows never overwritten). ☐ PASS ☐ FAIL

TC-B.4 — Statutory rates & thresholds

1. **Configuration** → **Statutory Rates**: view/add a rate (e.g. VAT %, PAYE bands). 2. **Thresholds**: view the petty-cash FD threshold (per currency). **Expected:** **EXPECT** ✓ Both are editable, effective-dated config (nothing hard-coded). ☐ PASS ☐ FAIL

TC-B.5 — Approval matrix

1. **Configuration** → **Approval Matrix**. 2. View the seeded rows (requisition→SiteMgr then FD>5000; budget→OD+FD parallel; etc.). 3. Add/adjust a band. **Expected:** **EXPECT** ✓ Matrix drives the approval engine (verified later in Part E). ☐ PASS ☐ FAIL

TC-B.6 — Delegation, Danger Rules, Settings, Entities

1. **Delegation**: create a delegation from `fd` to `od` for a date range. 2. **Danger Rules**: toggle a rule on/off. 3. **Settings/Lookups**: add a lookup value (e.g. a new currency `ZAR`). 4. **Entities**: view "AES Zimbabwe"; create a 2nd entity (e.g. "AES Botswana", country BW, currency BWP). **Expected:** **EXPECT** ✓ All configurable. (Delegation is exercised in TC-6.5; entity in Part N.) ☐ PASS ☐ FAIL

TC-B.7 — Audit log

1. **Configuration** → **Audit Log**. 2. Search by actor/table. 3. Find one of the changes you made above. **Expected:** **EXPECT** ✓ Every change is logged with actor + timestamp. Try to find an **edit/delete** control — there should be none (append-only). ☐ PASS ☐ FAIL

PART C — Financial core (data entry + live dashboards)

Admin console (Finance/Sales groups) or Web app. Use `admin`, `fd`, `fo`, or `opsstaff`.

TC-C.1 — Clients

1. **Sales & CRM** → **Clients**. 2. Create a client (name, contact, VAT no.). 3. Edit it. **Expected: EXPECT** ✓ Client saved; appears in order/contract drop-downs. ☐ PASS ☐ FAIL

TC-C.2 — Contracts + variance

1. **Contracts**. 2. Create a contract (value ex-VAT, start/end). 3. Open it. **Expected: EXPECT** ✓ Monthly budget, invoiced-to-date, expected-to-date and **variance** compute automatically (red when over-claimed). ☐ PASS ☐ FAIL

TC-C.3 — Orders + computed health/profit (G16)

1. **Orders**. 2. Create an order (client, value ex-VAT, closing date). 3. Open the order detail. **Expected: EXPECT** ✓ VAT @15.5%, total incl VAT, **health status**, **profit/margin**, outstanding and spent-to-date are shown/computed on the order (not just on a dashboard). ☐ PASS ☐ FAIL

TC-C.4 — Partial order servicing (G18)

1. Open an order. 2. Add two **milestones** (e.g. 50% each), mark one complete. **Expected: EXPECT** ✓ Serviced % shows ~50%; the order health becomes **PARTIALLY SERVICED**. ☐ PASS ☐ FAIL

TC-C.5 — Order expenses / general expenses / overheads / loans / claims

1. Add an **order expense** (category, amount, VAT-paid flag). 2. Add a **general expense**. 3. Add an **overhead** (salaries, PAYE due/paid). 4. Add a **loan** linked to an order (principal, weekly rate). 5. Add a **contract claim**. **Expected: EXPECT** ✓ Each saves; input VAT + loan interest compute; the order's spent-to-date/profit update. ☐ PASS ☐ FAIL

TC-C.6 — Tax ledger reflects operations

1. After the entries above, open the tax view / Command Centre tax exposure. **Expected: EXPECT** ✓ Output VAT (orders+claims) – input VAT (expenses/overheads) + brought-forward = net VAT payable; PAYE payable shown. (Values match the workbook after the demo import — see TC-4.2.) ☐ PASS ☐ FAIL

PART D — Data import & migration parity

TC-D.1 — Import the operational workbook

Role: `admin` or `fd` · **Where:** Admin → **Configuration** → **Data Import** 1. Select **All workbooks**. 2. Click **Import**.

Expected: **EXPECT** ✓ A summary table shows rows created/updated per table (clients, contracts, orders, receipts, expenses, overheads, loans, claims, employees, payroll runs/lines, statutory returns). No errors. ☐ PASS ☐ FAIL

TC-D.2 — Migration parity → ACT verdict

1. On the same page, **Migration parity** section → **Re-run**. **Expected:** **EXPECT** ✓ Big "**ACT** ✓" health verdict (expected ACT). All checks **PASS** — Cash received ≈ \$52,463; Outstanding receivables ≈ \$113,972; Operating profit \$90,250; Net profit after loans \$76,195.71; Total tax liability \$52,062.83; Loan balance \$44,054.29. Overall "**Migration parity achieved**". ☐ PASS ☐ FAIL

TC-D.3 — Dashboards light up

1. Go to the admin **Dashboard** and the **Command Centre** (web app). **Expected:** **EXPECT** ✓ Non-zero figures everywhere (10 orders, revenue/profit, receivables, tax). The command-centre verdict reflects the imported data. ☐ PASS ☐ FAIL

PART E — Approvals engine & cash workflows (cross-role money flows)

This is the heart of the system. Each flow spans multiple roles — switch accounts as indicated.

TC-E.1 — Requisition: raise → approve → disburse

1. As **opsstaff** (web app → **Requests** → **Requisition**): raise a requisition (purpose, amount **\$300** USD, required-by date, optional receipt). Submit.
2. As **sitemgr** (Approvals inbox): the item appears → **Approve** (step 1).
3. Because $\$300 < \5000 , no FD step is needed → status becomes **Approved-Ready-to-Pay** (funds sufficient) or **Approved-Pending-Funds** (if short).
4. As **fo**: open the item → **Disburse** (choose a source account, capture a reference). **Expected: EXPECT ✓**
Status lifecycle Draft→Submitted→Approved→Ready-to-Pay/Pending-Funds→Disbursed; a ledger entry posts; requester + finance are notified. ☐ PASS ☐ FAIL

TC-E.2 — Requisition over the band needs FD (sequential chain)

1. As **opsstaff**: raise a requisition for **\$8,000**. Submit.
2. As **sitemgr**: Approve (step 1). 3. As **fd**: Approve (step 2). **Expected: EXPECT ✓** Two-step chain; only after **both** does it become approved. Notifications fire at each step. ☐ PASS ☐ FAIL

TC-E.3 — Approve-first / Pending-Funds

1. Raise a requisition larger than available cash (e.g. **\$500,000**). Approve it through the chain. **Expected: EXPECT ✓** It is **approved on merit** and flagged **Approved-Pending-Funds** with the shortfall; it does **not** block on funds. (A daily job re-tests it.) ☐ PASS ☐ FAIL

TC-E.4 — Travel allowance + retirement (+ overspend)

1. As **opsstaff**: raise a **Travel** request (destination, dates → per-diem auto-computed). Submit → approve (sitemgr/fd) → disburse.
2. As **fo**: **Retire** the advance: enter unspent cash **less** than the advance → refund due.
3. Retire another with spend **more** than the advance → **refund owed** to the traveller (overspend) is captured. (G25) **Expected: EXPECT ✓** Per-diem auto-calc; retirement reconciles both refund-due and refund-owed. ☐ PASS ☐ FAIL

TC-E.5 — Petty cash (below vs above threshold)

1. As **fo**: create a petty-cash **float** for a site+currency. (Try to create a second float for the same site+currency → **MUST BLOCK X** blocked, one-per-site-per-currency, G25.)
2. As **clerk**: record a **withdrawal below** the FD threshold (e.g. \$40) with a receipt → As **sitemgr**: confirm it → posts immediately.
3. As **clerk**: record a withdrawal $\geq$ **threshold** (e.g. \$150) → it routes to **fd** for approval before cash leaves. **Expected: EXPECT ✓** Below-threshold posts on site-manager confirm; at/above threshold requires FD. **Separation:** the voucher raiser cannot confirm their own voucher. ☐ PASS ☐ FAIL

TC-E.6 — Petty cash conversion + reconciliation + conversions report (G25)

1. As **fo**: record a USD↔ZWG **conversion** (both legs, achieved rate). 2. Record a **cash count** with a variance beyond tolerance → alert to FD + float locks. 3. As **fd**: unlock. 4. Open the **conversions report** (cumulative gain/loss per site). **Expected: EXPECT ✓** Conversion stores variance vs official; out-of-tolerance count locks the float; conversions report totals gain/loss per site. ☐ PASS ☐ FAIL

TC-E.7 — Budgets: dual authorisation (OD + FD, parallel)

1. As **fo** : create a **Budget** with line items → Submit.
2. As **od** : approve. 3. As **fd** : approve. **Expected: EXPECT ✓** Budget becomes **Active only after BOTH** OD and FD approve (parallel). Either **Return** restarts the cycle. Actual spend tracks against lines with % consumed. ☐
PASS ☐ FAIL

TC-E.8 — Director withdrawal: co-approve → post → complete

1. As **director** : raise a **Director Withdrawal** (amount, destination, reason) → Submit.
 2. As **fd** (or another director): co-approve.
 3. On approval it posts to the ledger as "**Posted — Awaiting Transfer**" (visible immediately in cash position).
 4. As **fo** : select transfer method (EFT/RTGS), capture reference → **Complete. Expected: EXPECT ✓** Appears in the ledger/financial summary on approval (before transfer); completion needs method + reference; the full trail is auditable. ☐ PASS ☐ FAIL
-

PART F — Segregation of duties & security (negative tests — blocking = PASS)

TC-F.1 — No self-approval

1. As **fd** : raise a requisition. 2. As the **same fd** , open the Approvals inbox. **Expected: MUST BLOCK x** Your own request is **not** approvable by you (absent from your inbox; a direct approve attempt is forbidden). ☐ PASS ☐ FAIL

TC-F.2 — Decide authorisation (role required) (G3)

1. As **opsstaff** (who is not an approver on the chain), obtain a pending approval item and attempt to decide it (via the API `/v1/approvals/:id/decide` , or a URL). **Expected: MUST BLOCK x 403 Forbidden** — only a holder of the step's role (or an active delegate) can decide. ☐ PASS ☐ FAIL

TC-F.3 — Auditor is read-only

1. As **auditor** : browse dashboards, orders, payroll (masked), audit log. 2. Attempt any create/edit/approve. **Expected: EXPECT ✓** Can read everywhere; **MUST BLOCK x** every write/approve is blocked. ☐ PASS ☐ FAIL

TC-F.4 — Site scoping (Mimosa clerk can't see Unki)

1. As **clerk** (Mimosa): view timesheets/orders/site data. **Expected: EXPECT ✓** Only **Mimosa** data is visible; **MUST BLOCK x** no Unki/Zimplats data. ☐ PASS ☐ FAIL

TC-F.5 — Payroll separation of duties

1. As **fo** : prepare/compute a payroll run. 2. Attempt to also **approve** it. **Expected: MUST BLOCK x** Preparer ≠ approver — the FO cannot approve their own run; **fd** must. Once approved, the run **locks** (no edits). ☐ PASS ☐ FAIL

TC-F.6 — Audit log is append-only

1. Confirm there is no UI to edit/delete audit rows. (Deeper: a DB `UPDATE / DELETE` on `audit_log` is rejected at the database — G6.) **Expected: MUST BLOCK x** Audit entries cannot be altered or removed. ☐ PASS ☐ FAIL

PART G — Command Centre & danger alerts

TC-G.1 — Command Centre panels

Role: fd / director / od · **Where:** Web app → **Command Centre** 1. Open it (after the demo import). **Expected:** **EXPECT** ✓ All panels render: Cash position (per account + USD-equiv official/street), Money in vs out, Debt & interest watch, Orders vs payroll/expenses (coverage ratio), Receivables ageing, Tax exposure, Pending obligations, and the **Health verdict** (HEALTHY/WATCH/**ACT**). ☐ PASS ☐ FAIL

TC-G.2 — Danger alerts + acknowledgement

1. View the alert feed. 2. Acknowledge an alert. **Expected:** **EXPECT** ✓ Alerts listed by severity; acknowledging records who/when and drops it from the active feed (DANGER alerts repeat until acknowledged). Directors always receive DANGER. ☐ PASS ☐ FAIL

TC-G.3 — Danger rules fire

1. As **admin**: in Danger Rules confirm rules like cash_runway, coverage_ratio, zimra_overdue, **payroll_coverage**, petty_cash_variance, concentration_risk, conversion_loss are enabled. **Expected:** **EXPECT** ✓ Rules are configurable; after the import, relevant rules evaluate and surface on the Command Centre. (G8) ☐ PASS ☐ FAIL

PART H — Timesheets

TC-H.1 — Capture a daily grid

Role: **clerk** (Mimosa) · **Where:** Web app → **Timesheets** (or mobile) 1. Open the current period for Mimosa. 2. Enter daily hours for a few employees across the 5 categories (Normal, OT1.5, OT2.0, Underground, Night). **Expected:** **EXPECT** ✓ Grid saves; monthly manhours roll up per employee. ☐ PASS ☐ FAIL

TC-H.2 — Submit → site-approve → lock

1. As **clerk**: submit the period. 2. As **sitemgr**: approve it → it **locks**. **Expected:** **EXPECT** ✓ Lifecycle Open→Submitted→Site-Approved→Locked; locked periods reject edits. ☐ PASS ☐ FAIL

TC-H.3 — Reopen-unlock (G22)

1. As **sitemgr** (or **admin**): reopen the locked period. **Expected:** **EXPECT** ✓ Period reverts to **Open** (audited). **MUST BLOCK X** If a payroll run already consumed it, reopen is blocked unless an admin forces it. ☐ PASS ☐ FAIL

TC-H.4 — Client manhours export

1. As **fo** / **sitemgr**: export the manhours register (XLSX) for a site/month. **Expected:** **EXPECT** ✓ A downloadable manhours file per site. ☐ PASS ☐ FAIL

PART I — Payroll → Statutory Returns (cross-role)

TC-I.1 — Run payroll

Role: **fo** (prepare) then **fd** (approve) · **Where:** Admin → **People & Payroll** → **Payroll** 1. **As fo** : open a run for a site + month (e.g. Mimosa) → **Compute**. 2. Review the paysheet (per-currency PAYE + AIDS levy, NSSA, ZIMDEF, NEC, MIPF, Nyaradzo, net USD/ZWG). 3. **Submit** for approval. 4. **As fd** : **Approve** → run locks. **Expected:** **EXPECT** ✓ Statutory stack computes per currency; net pay looks right; approval locks the run and posts salaries/employer cost to the ledger + overheads. ☐ PASS ☐ FAIL

TC-I.2 — Payroll outputs

1. From an approved run, download: **Payslips (PDF)**, **Bank schedule**, **Sage journal (CSV)**. **Expected:** **EXPECT** ✓ One payslip page per employee; bank schedule per bank/currency; balanced Sage journal. ☐ PASS ☐ FAIL

TC-I.3 — Statutory obligations auto-created (G22)

1. After approving the run, open **Compliance calendar**. **Expected:** **EXPECT** ✓ Statutory obligations (PAYE/NSSA/ZIMDEF/NEC/MIPF/Nyaradzo) are **auto-generated** with due dates (no manual step). ☐ PASS ☐ FAIL

TC-I.4 — Returns Hub

Role: **fd / fo** · **Where:** Admin → **Finance** → **Returns Hub** 1. Pick the run's period → the payroll statutory rows appear (or click **Post from payroll**). 2. Click **Post VAT** for a period. 3. **Remit** a row (date, amount, reference, proof). **Expected:** **EXPECT** ✓ One row per tax type/period with due/paid/balance/deadline/status (DUE→OVERDUE auto). Remittance leaves a balance if partial. ☐ PASS ☐ FAIL

TC-I.5 — Withholding tax (both directions)

1. **WHT payable:** record a supplier payment with *no tax clearance* → the configured WHT % is withheld automatically and a liability row is created. 2. **WHT suffered:** record WHT a client withheld from AES → accumulates as a credit; certificate tracked. **Expected:** **EXPECT** ✓ Auto-withhold when clearance absent; suffered WHT tracked as a credit. ☐ PASS ☐ FAIL

PART J — Back-pay & Acting allowances

TC-J.1 — Back-pay batch

Role: fd / admin · **Where:** Admin → Finance → Payroll Adjustments → Back-pay 1. Create a batch: new NEC rates effective a past date, affected periods (e.g. Mar–May). 2. Review per-employee old-vs-new-vs-difference workings. 3. **Submit** → as fd approve. **Expected:** EXPECT ✓ Difference computed per employee×period; on approve it emits a back-pay earning that the **next payroll run picks up** as a line (verify in TC-I.1 on a later run). ☐ PASS ☐ FAIL

TC-J.2 — Acting allowance

1. **Acting allowances** tab: create an acting assignment (employee, acting position, dates, fixed or % basis). 2. Try to create an **overlapping** assignment for the same employee → **MUST BLOCK X** blocked. 3. Submit → approve. **Expected:** EXPECT ✓ Pro-rated across the run, own payslip line; overlaps blocked; appears on the acting register. ☐ PASS ☐ FAIL
-

PART K — CRM & Marketing (business development)

TC-K.1 — CRM pipeline → convert

Role: opsstaff / od / admin · **Where:** Web app → **CRM** (or Admin → Business Dev) 1. Create an **organisation** + **contact**. 2. Log an **interaction**. 3. Create an **opportunity** and move it through the pipeline (Contact→Qualified→...→Won). 4. On **Won**, use **Convert** → creates an Order or Contract. **Expected:** **EXPECT** ✓ Pipeline stages enforced; one-click conversion creates a linked order/contract; conversion analytics update. ☐ PASS ☐ FAIL

TC-K.2 — Marketing campaigns + funnel

Role: fd / od / admin · **Where:** Web/Admin → **Marketing** 1. Create a **campaign** (budget, dates). 2. Add **channels** (a social one + a **Flier** — note the auto flier code). 3. Add weekly **metrics**. 4. Capture **responses/leads** ("how did you hear?"), move a lead status, **convert** to an opportunity. 5. Open **Analytics**. **Expected:** **EXPECT** ✓ Funnel per campaign, cost-per-lead leaderboard, flier-vs-social comparison. ☐ PASS ☐ FAIL

PART L — Boards (incl. Director-Confidential) — key security test

TC-L.1 — Create boards

Role: `director` · **Where:** Web/Admin/Mobile → **Boards** 1. Create a **TEAM** board ("Ops Tasks"). 2. Create a **DIRECTOR-CONFIDENTIAL** board ("Project Falcon"). 3. Add lists, cards, checklist items, comments. **Expected:** **EXPECT** ✓ Both created; confidential board shows a red **DIRECTOR-CONFIDENTIAL** chip. ☐ PASS ☐ FAIL

TC-L.2 — Confidential is hidden from non-directors (server-side)

1. As `opsstaff` (or `clerk`, or even `admin`): open Boards. **Expected:** **MUST BLOCK x** The **Ops Tasks** (team) board is visible, but **Project Falcon** (confidential) is **completely absent** — even for the System Administrator. (It's filtered server-side, not just hidden in the UI.) ☐ PASS ☐ FAIL

TC-L.3 — Director sees confidential

1. As `director` (or `fd`, any `_DIRECTOR`): open Boards. **Expected:** **EXPECT** ✓ **Both** boards visible, including Project Falcon. ☐ PASS ☐ FAIL

TC-L.4 — Reclassify (graduation)

1. As `director`: reclassify Project Falcon → TEAM. **Expected:** **EXPECT** ✓ It becomes visible to everyone; the change is audited. ☐ PASS ☐ FAIL

PART M — Projects / WBS

TC-M.1 — Build a project

Role: `sitemgr` / `od` / `admin` · **Where:** Web/Admin → **Projects** 1. Create a project (site, contract). 2. Add a **phase**, then **tasks/sub-tasks** with weights and dates. **Expected:** **EXPECT** ✓ WBS tree renders (phases › tasks › subtasks). ☐
PASS ☐ FAIL

TC-M.2 — Update progress → roll-up

1. Update a task's % (slider) + one-line note; mark another complete. **Expected:** **EXPECT** ✓ Progress **rolls up** (weighted) to the phase and project; the portfolio shows the single % and a **RAG** flag + days ahead/behind. ☐
PASS ☐ FAIL

TC-M.3 — Mobile-light update

Role: `sitemgr` on **mobile** → **Projects** 1. Tap a task → slide %, add a note, attach a photo. **Expected:** **EXPECT** ✓ Light update flow works; photo attaches; roll-up reflects. ☐ PASS ☐ FAIL

TC-M.4 — Templates

1. Create a project **from a template**. **Expected:** **EXPECT** ✓ The template explodes into a ready WBS. ☐ PASS ☐
FAIL

PART N — Site Reports, SHE & Multinational

TC-N.1 — Monthly site report

Role: `sitemgr` / `od` · **Where:** Web/Admin → **Site Reports** 1. Open a period (site, contract, month). 2. Fill sections (Admin/Operations/Finance/PPE/Withdrawals/Logistics/SHE/Other) + narrative. 3. **Submit** → it locks. **Expected:** **EXPECT** ✓ Sections editable; submit locks & archives; late submission flags the timeliness KPI. ☐ PASS ☐ FAIL

TC-N.2 — Performance vs target (RAG)

1. Set KPI targets + actuals → view **Performance** and **Cross-site**. **Expected:** **EXPECT** ✓ Each KPI scores vs target with RAG; a weighted site score; cross-site comparison. ☐ PASS ☐ FAIL

TC-N.3 — SHE capture (G21)

Role: `clerk` / `sitemgr` · **Where:** Admin → **Operations** → **SHE** 1. **Log** a SHE record (Incident / Near-miss / Toolbox-talk, site, severity, LTI toggle). 2. **As** `sitemgr` : update status / add investigation. 3. View **stats**. **Expected:** **EXPECT** ✓ Records saved; stats summary (counts by type, LTI count, open investigations). ☐ PASS ☐ FAIL

TC-N.4 — Multinational (2nd entity) (G4)

Role: `admin` · **Where:** Admin → **Configuration** → **Entities** 1. Confirm "AES Zimbabwe" exists. 2. Create "AES Botswana" (country BW, currency BWP). 3. Add a public holiday. 4. Open its summary. **Expected:** **EXPECT** ✓ A second entity with its own country/currency exists; existing data stays keyed to AES Zimbabwe. ☐ PASS ☐ FAIL

PART O — Notifications & Reports

TC-O.1 — Notification preferences (G24)

1. **Web app** → **Profile** → **Notification preferences**: toggle push/email on/off. 2. **Admin** → **Configuration** → **Notifications**: view the severity→channel matrix (in-app always; push/email from WATCH; Teams for DANGER).
Expected: EXPECT ✓ Prefs save; the matrix is shown. (*Actual push/email/Teams delivery only fires when SMTP/Teams/FCM creds are configured — otherwise in-app only.*) ☐ PASS ☐ FAIL

TC-O.2 — Reports/exports

1. **Admin** → **Finance** → **Reports**: download Financial Summary (XLSX), Cashflow (XLSX), Manhours (XLSX), Payslips (PDF), Sage journal (CSV). **Expected: EXPECT** ✓ Each downloads with real data. ☐ PASS ☐ FAIL
-

PART P — Mobile app (by role)

Android emulator, logged in against the running API.

TC-P.1 — Home dashboard is role-aware

1. Log in as different roles. **Expected: EXPECT** ✓ Home tiles differ by role; a **DANGER banner** shows when alerts are active (directors/FD). ☐ PASS ☐ FAIL

TC-P.2 — Approvals with biometric on money items

Role: `fd / sitemgr` 1. Open Approvals → approve a **money** item. **Expected: EXPECT** ✓ A biometric/confirm prompt appears for money items; declining cancels without calling the API. ☐ PASS ☐ FAIL

TC-P.3 — Requests + camera receipt (offline)

Role: `clerk / opsstaff` 1. Raise a requisition + capture a receipt photo. 2. Turn on airplane mode, raise another. **Expected: EXPECT** ✓ Receipt attaches; the offline one **queues** and **syncs** when back online (pending-sync banner). ☐ PASS ☐ FAIL

TC-P.4 — Mobile Timesheets / Projects / Boards / Command Centre / Director

1. Exercise each new mobile screen (Timesheets grid, Projects light-update, Boards incl. confidential visibility by role, Command Centre verdict, Director actions). **Expected: EXPECT** ✓ Each screen loads its data and its primary action works. ☐ PASS ☐ FAIL

Appendix A — Reset / re-run

- **Re-import is idempotent** — re-running Data Import updates rather than duplicates.
- To reset a workflow test, raise fresh items (there are no destructive delete endpoints for approved financial data by design).
- If a role can't log in, an admin can reset the password in **Users & Roles** (or re-create the test user).

Appendix B — Known open items (expected gaps — do NOT raise as bugs)

All are documented in `docs/GAP_ANALYSIS.md`. Read this before testing so you don't log expected gaps as defects.

B.1 — Requires external / third-party setup (cannot be tested until provisioned)

These features are **built or wired, but inert** until an external service + its credentials are supplied. Until then the behaviour below is **expected, not a bug** — skip or note "blocked on external setup".

Item	External dependency (who must provide it)	What you'll see / test impact
G13 — Fabric / Power BI / "chat-with-your-data" AI analytics	Microsoft Fabric tenant + paid capacity (Microsoft 365 / Azure)	No BI dashboards, no Copilot/Data-Agent. Skip entirely — nothing to test in-app.
Email alerts & notifications (part of G7)	SMTP server creds, or Microsoft 365 / Graph (Outlook)	Emails do not arrive. In-app notifications do work. Set <code>MAIL_*</code> env to activate.
Teams alerts (part of G7)	Microsoft Teams Incoming Webhook URL	No Teams messages. In-app works. Set <code>TEAMS_WEBHOOK_URL</code> to activate.

Item	External dependency (who must provide it)	What you'll see / test impact
Mobile push (FCM) + G28	Google Firebase project + service-account JSON (+ device-token registration)	No push to the phone. In-app feed + deep-link routing work. Set FCM_SERVICE_ACCOUNT_JSON + build the token store to activate.

This is why **TC-O.1** notes that push/email/Teams delivery is inactive: the transports are implemented and route by severity, but stay silent (in-app only) until the credentials above are configured.

B.2 — Not yet built (internal — planned, don't file as bugs)

- **G15** — some formatted/scheduled **exports**: bank-schedule *file*, ZIMRA returns *pack*, per-client report. (*The scheduled-email delivery of reports also depends on SMTP from B.1.*)
- **G26** — **MFA enforcement** for FD/Directors/Admin (login works; MFA is not yet enforced).
- **G27** — **backups / disaster recovery** — a deployment/hosting concern (managed-Postgres PITR), not application behaviour.
- Minor: in-app **help pages**; a residual hard-coded USD/ZWG assumption in a few reporting/danger panels (adding a third currency such as ZAR may not total everywhere yet).

Appendix C — Coverage checklist (tick when the whole part passes)

☐ A Auth/RBAC
 ☐ B Admin config
 ☐ C Financial core
 ☐ D Import/parity
 ☐ E Approvals/cash
 ☐ F Segregation/security
 ☐ G Command Centre
 ☐ H Timesheets
 ☐ I Payroll>Returns
 ☐ J Back-pay/Acting
 ☐ K CRM/Marketing
 ☐ L Boards
 ☐ M Projects
 ☐ N Site Reports/SHE/Entities
 ☐ O Notifications/Reports
 ☐ P Mobile